

Code-Layout, Readability and Reusability

Date	3 July 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	Complies with PEP 8 standards.
2	Proper File Structure	Files and folders are logically organized.	Yes	Standard Flask structure (templates, static, model).
3	Meaningful Variable Names	Variables reflect their purpose clearly.	Yes	Used N, P, K, ph, temperature, etc. explicitly.
4	Function / Method Names	Functions are descriptively named.	Yes	Used recommend(), suitability(), get_crops(), etc.
5	Code Comments	Inline and block comments explain logic.	Yes	Detailed docstrings and block comments included.
6	Modular Design	Code is split into reusable functions/modules.	Yes	Separate training scripts, backend routing, and stats layers.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	No redundant functions, optimized imports.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	Uses try-except blocks for data parsing and range validations.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	model.pkl	Python / Scikit-learn	Flask Backend (app.py)	High

2	crop_stats.json	JSON Data	Suitability Engine & Charts Dashboard	High
3	style.css	CSS3 / Custom properties	All UI views (home, recommend, suitability, research)	High
4	base.html	Jinja2 HTML Template	Renders navbar, header, footer for all sub-pages	High
5	get_crops()	Python helper function	Used in suitability and research routes	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Logically divided templates, static assets, and modular python code.
Code Readability & Formatting	5	Consistent naming conventions, clear indentation, and descriptive docstrings.
Code Comments & Documentation	5	Comprehensive comments explaining ML validations and suitability algorithms.
Reusability & Modularity	5	Reusable data models, templates, and UI styling sheets.
Error Handling & Validation	5	Robust check blocks for out-of-bounds inputs and non-negative parameters.